

Thư Âu

Thesis Report.docx (1).pdf

 Thesis
 Thesis_HK2_25-26
 turnitin@hcmiu.edu.vn

Document Details

Submission ID**trn:oid:::1:3563809535****Submission Date****May 8, 2026, 2:12 PM GMT+7****Download Date****May 8, 2026, 2:35 PM GMT+7****File Name****Thesis_Report.docx_1_.pdf****File Size****2.4 MB****77 Pages****12,408 Words****78,493 Characters**

23% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **50 AI-generated only 23%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



VIETNAM NATIONAL UNIVERSITY
OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A Cross-platform Plugin System for Phishing Link Detection and Warning

By

Au Nguyen Nhat Thu

Advisor: Dr. Le Hai Duong

A thesis submitted to the School of Computer Science and Engineering

in partial fulfillment of the requirements for the degree of

Bachelor of Information Technology

Ho Chi Minh City, Vietnam

2026

A Cross-platform Plugin System for Phishing Link Detection and Warning

APPROVED BY: _____ ,

Nguyen Van A, Ph.D, Chair (*Example*)

**(Type Committee names beneath lines)*

(Typed Committee name here)

(Typed Committee name here)

(Typed Committee name here)

(Typed Committee name here)

THESIS COMMITTEE

(Whichever applies)

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Le Hai Duong, for his formal supervision, guidance, and administrative support throughout the thesis process. His oversight and feedback provided the necessary academic framework for this study.

TABLE OF CONTENTS

ACKNOWLEDGMENTS.....	3
TABLE OF CONTENTS.....	4
LIST OF FIGURES.....	6
LIST OF TABLES.....	7
ABSTRACT.....	9
CHAPTER I.....	10
INTRODUCTION.....	10
1.1. Background.....	10
1.2. Problem Statement.....	10
1.3. Scope and Objectives.....	12
1.3.1. Scope.....	12
1.3.2. Objectives.....	13
1.4. Assumption and Solution.....	13
1.4.1. Assumption.....	13
1.4.2. Proposed solution.....	14
1.5. Structure of thesis.....	15
CHAPTER II.....	17
LITERATURE REVIEW RELATED WORK.....	17
2.1. Overview of phishing detection.....	17
2.2. Cross-platform system.....	18
2.2.1. Web plugin.....	18
2.2.2. Mobile app detection.....	18
2.3. URL Heuristic Detection Methods.....	19
2.4. Threat Intelligence and Domain Analysis.....	20
2.5. Machine learning for phishing detection.....	21
2.6. LLM.....	22
2.6.1. Logo detection.....	23
2.6.2. Content detection.....	23
2.6.3. Meta data and content analysis.....	24
2.7. Scoring system.....	24
CHAPTER III.....	26
Methodology.....	26
3.1. System Overview and Architecture.....	26
3.1.1. System overview.....	26
3.1.2. Architecture.....	27
3.2. Multi-layer phishing detection approach.....	29
3.2.1. Rationale for Hybrid Detection.....	29
3.2.2. Rule-Based Heuristic Detection (Layer 1).....	29
3.2.3. Threat Intelligence (Layer 2).....	31
3.2.4. Domain and SSL Analysis (Layer 3).....	31
3.3. Machine Learning Model Development.....	33
3.3.1. Prepared dataset.....	33

3.3.2. Preprocessing and feature engineering.....	33
3.3.2.1 Feature engineering.....	33
3.3.2.2. Preprocessing.....	34
3.3.3. Model selection and training.....	35
3.3.4. Model evaluation.....	36
3.4. Large language models (LLM).....	38
3.4.1. Page content retrieval.....	38
3.4.2. Logos and Favicon analysis.....	39
3.4.3. Page content analysis.....	40
3.4.4 Meta data analysis.....	41
3.4.5. Cross-matching.....	42
3.5. Weighted Scoring Engine and Threshold Calibration.....	43
3.5.1. Score aggregation formula.....	43
3.5.2. Threshold calibration.....	43
3.6. Tool and technology.....	44
3.6.1. Browser Extension (Desktop).....	44
3.6.2. Backend Service.....	44
3.6.3. Machine Learning.....	44
3.6.4. Mobile Application.....	45
3.6.5. Development Tools.....	45
CHAPTER 4.....	46
IMPLEMENT AND RESULTS.....	46
4.1 Backend API Server Implementation.....	46
4.1.1. Run the backend.....	46
4.1.2 API Endpoints.....	47
4.1.3 Request and Response Structure.....	48
4.1.4 Module Loading and Initialization.....	49
4.2. Machine learning models result.....	49
4.2.1. Models comparison.....	49
4.2.2. Feature importance analysis.....	51
4.2.3 Model Serialization.....	53
4.3 Threshold Calibration and Scoring System.....	54
4.3.1. Score Distribution Analysis.....	54
4.4.2 Confusion Matrix Analysis at Multiple Thresholds.....	56
4.5. LLM Analysis Result.....	58
4.5.1. Legitimate websites.....	58
4.5.2. Typosquatting detection.....	58
4.5.3. Brand-impersonation.....	59
4.5.4. Maximum risk.....	59
4.6 Implementing Extensions.....	61
4.6.1 URL Interception Mechanism.....	61
4.6.2 Warning Page.....	62
4.6.3 Fail-Safe Mode.....	62
4.7 React Native Mobile Application Deployment.....	62
4.7.1 URL Blocking Simulation.....	62

4.7.2 Warning Modal.....	63
4.7.3 Consistency Verification.....	63
CHAPTER 5.....	64
DISCUSSION AND EVALUATION.....	64
5.1. Evaluation metrics of machine learning.....	64
5.1.1. Key metrics.....	64
5.1.2. Machine learning evaluation.....	65
5.2. Optimize threshold.....	65
5.3. Evaluate LLM integration.....	66
5.4. Discussion about the advantage and disadvantage of hybrid detecting system.....	67
CHAPTER 6.....	68
CONCLUSION AND FUTURE WORK.....	68
6.1. Conclusion.....	68
6.1.1. What We Did.....	68
6.1.2. What Has Been Accomplished.....	68
6.1.3. Why It Matters.....	69
6.2. Future work.....	70
REFERENCES.....	71
APPENDIX.....	75

LIST OF FIGURES

Figure 3.1.1: architecture of phishing detection system.....	28
Figure 3.4.1: fetch_page() function.....	40
Figure 3.4.2: extract logo from url function.....	40
Figure 3.4.3: analyze logo function.....	41
Figure 3.4.4: extract page content function.....	42
Figure 3.4.5: analyze content function.....	42
Figure 3.4.6: extract metadata function.....	42
Figure 4.1.1: Running the backend server.....	48
Figure 4.1.2: response from an endpoint for phishing URL.....	49
Figure 4.1.3: backend diagnostic summary.....	50
Figure 4.2.1: XGBoost top feature importance.....	52
Figure 4.2.2: Random Forest top feature importance.....	52
Figure 4.2.4: Decision tree top feature importance.....	53
Figure 4.3.1: Total Score Distribution.....	54
Figure 4.3.2: Rules Score Distribution.....	55
Figure 4.3.3: Machine Learning Probability Distribution.....	56
Figure 4.4.1: Confusion Matrices at Thresholds 30, 40.....	57
Figure 4.4.2: Confusion Matrices at Thresholds 50, 60.....	57
Figure 5.2.1: Threshold visualization.....	66
Figure A.1: Detection Performance at Key Thresholds.....	75
Figure A.2: warning page of web browser.....	76
Figure A.3: warning popup for safe URL in mobile.....	76
Figure A.4: warning popup for suspicious URL in mobile.....	77

LIST OF TABLES

Table 3.2.1: Domain age table with risk level and its penalty point.....	32
Table 3.4.1: cross-match penalty.....	42
Table 4.1.1: API end point.....	47
Table 4.2.1: Performance Comparison of Trained Classifiers.....	49
Table 4.5.1: LLM Analysis Test Results.....	58

ABSTRACT

In many recent years, phishing attacks have become increasingly sophisticated. Hackers often exploit vulnerable websites which traditional mechanisms cannot detect. Moreover, nowadays one user can use many devices such as laptops, mobile phones, and iPads for working, studying or entertaining so the possibility that users incidentally access phishing links also increases. To address these issues, they need a system that can detect whether the website is secure or not. To verify if a website is safe, users need to examine whether the website's URL is secure. Hence, the solution to solve this problem is to develop a cross-platform system for detecting phishing links. The system has a multi-layered hybrid engine that integrates rule-based analysis, threat intelligence, and machine learning (ML). Besides, it used Large Language Models (LLMs) to check a website's logo and compare content to the real websites. The system is evaluated by applying models to train a dataset containing both benign and malicious URLs. From the training result, we can find ways to optimize the risk scoring threshold to achieve the highest accuracy, thereby protecting users from phishing threats via fake links and pages.

CHAPTER I

INTRODUCTION

1.1. Background

In many recent years, phishing attacks have become one of the most common cyber threats since the number of victims on social media platforms, email, messaging, etc., is increasing rapidly. According to APWG, in 2025, there were 3.8 million phishing attacks, an increase of 0.04 million attacks compared to 2024. Social networks, software-as-a-service (SaaS), and email are the targeted categories that easily get attacked [1]. Hackers exploit the ability to distinguish between real and fake websites of users; they use sophisticated methods such as fake domain names, changing a single letter or symbol in URLs, etc., to create fake websites that mimic legitimate ones. Then they send these URLs to users' email or post them on social media platforms. Because users cannot distinguish between real websites and fake ones, they easily trust and click on those websites, providing their personal information when requested. However, traditional detection methods are struggling to prevent these fraudulent activities, as these URLs and websites are more and more sophisticated, hard to detect if systems only apply simple methods.

1.2. Problem Statement

- Limitations of existing platform:

Normally, in the market, URLs checking platforms are built as a website, so when users search a domain in the search bar or click any links from their social media accounts or email, the platform does not automatically check and display a warning popup to notify them about the risk. Instead, if users want to check it, they have to perform these steps manually: copying the URL, accessing checking websites, pasting the link there, and requesting the system to analyze it. This process not only disrupts the user experience but also forces users to constantly switch between different tabs. It is time-consuming and leads users not to use URL checking tools frequently, thereby reducing the effectiveness of solutions for detecting fake or malicious websites in practice.

- Cost limitations:

Costs are a major obstacle to current phishing detection systems in the market nowadays. Most free URL detection web or mobile checking tools allow users to use basic methods such as rule-based, heuristic, and simple reputation checks to check the websites. Meanwhile, advanced techniques like machine learning (ML), deep learning (DL), or large language models (LLMs) are only integrated into paid versions or subscription packages. Free systems' accuracy is about 85% to 90% [2]; however, when dealing with sophisticated fake websites or pages that closely resemble legitimate websites in terms of interface and content, it still struggles to detect them. In practice, users often prefer free solutions to save costs. Hence, cost is one of the reasons that prevents users from accessing advanced security solutions. Therefore, users need a cross-platform phishing detection system that not only has advanced detection methods and high accuracy but still ensures reasonable costs and affordability for users.

1.3. Scope and Objectives

1.3.1. Scope

The scope of this is focused on building a cross-platform system with these components:

- Web extension:

Checks URLs in real time when users access the website. If it is unsafe, block the website and display a warning page with detailed explanations and action options.

- Mobile app:

Check URLs via the search bar display warning popup if needed.

- Backend detection engine:

Handle all phishing detection logic such as rule-based, lexical analysis, threat intelligence, domain and SSL verification, machine learning, and LLM-based semantic and visual analysis.

- Evaluation

Uses a phishing dataset to evaluate the system.

Measures performance using metrics such as accuracy, precision, and recall.

Tests multiple times to optimize results.

1.3.2. Objectives

The goal of this project is to develop a cross-platform phishing detection system that analyzes URLs in real-time and ensures providing clear explanations for users. The main objectives include:

- Building a phishing detection system in two versions: web extension and plugin. The backend is hybrid, which combines multiple methods such as rule-based, threat intelligence, machine learning, and LLM.
- Integrating Large Language Models (LLMs) to identify website content and logos. It helps improve the detection of sophisticated fake pages.
- Designing a scoring mechanism to classify a safe and unsafe page.
- Developing a user-friendly alert interface that provides specific reasons for detecting phishing threats.
- Ensuring consistency across platforms (web and mobile) through a backend.

1.4. Assumption and Solution

1.4.1. Assumption

When users surf social media, discounted products always get attention from users, so they tend to click on links to explore detailed information. However, these websites are not legitimate e-commerce platforms; they are fake ones. After reading product advertisements, they decide to purchase items, so they will unknowingly provide their personal information, account details, or payment information, leading to the risk of data theft or financial fraud. Similarly, when searching for information about hospitals or medical services, etc., on the

internet, users may also mistakenly access fake websites, leading to the provision of sensitive information or the receipt of inaccurate information. Moreover, nowadays, for more convenience in working and studying, users use many electronic devices such as laptops, phones, and iPads. So the possibility of accidentally accessing a fake website is much higher. This risk has become significant since most users cannot distinguish between the real, safe website and legitimate and fake websites by reading its patterns. Phishing websites often have these patterns: Suspicious URL structure such as typo-squatting, unusual length, unclear domain information,... However, no single method can accurately detect all phishing instances; therefore, a combination of methods and applied machine learning and LLMs is needed for improved system effectiveness.

1.4.2. Proposed solution

To solve the problem, we built a cross-phishing detection system with these functions:

- Multi-layer architect

The system combines various methods such as rule-based and lexical analysis to quickly detect anomalies in URLs, threat intelligence to check URLs against known phishing databases, domain and SSL analysis to assess website trustworthiness, and machine learning to predict phishing probabilities based on existing datasets.

- User interface

When phishing sites are detected, show the warning page (for the web version) or warning popup (for the mobile version) with specific reasons. The warning page provides two options, which are “Proceed anyway” and “Back to safe page”. “Proceed anyway” allows users to access the site even if it is unsafe. “Back to safe page” will redirect users to default browsers.

- Cross-platform consistency

Both the web plugin and mobile app use the same backend. The final result needs to ensure that if users check the same URL, it will provide the same results across all platforms.

- LLM integration for detecting content and logos

The system integrates LLMs to perform logo recognition and analyze page content such as text and metadata to understand the websites' true purpose. Based on this information, the system compares the consistency between the URL, logo, and content and compares them with the brand's real websites to detect it accurately and effectively.

- Scoring system

A scoring system is a mechanism that assigns a risk score to each URL based on results from multiple detection techniques. The score starts from 0, and while checking, if it meets some factors such as suspicious URL structure, unmatched content, domain, inconsistency between logo and branding, etc., the score will be added up. If the total risk score exceeds a defined threshold, the URL is phishing. If the score is below the threshold, the URL is safe. This approach allows the system to not only classify safe, unsafe links but also explain why a website is deemed dangerous.

1.5. Structure of thesis

This report organize into 6 chapters:

- Chapter 1: Introduce the background and reason to building this system.
- Chapter 2: Review related work.
- Chapter 3: Detail methodology and system design.
- Chapter 4: Present the implementation and result.

- Chapter 5: Discuss about the evaluation.
- Chapter 6: Concludes the report and summarizes the progression with future implementation.

CHAPTER II

LITERATURE REVIEW RELATED WORK

2.1. Overview of phishing detection

Phishing attacks are one of the frauds that utilize the internet. Cybercriminals use this type of attack to manipulate users into revealing information such as financial data or personally identifiable details. There are many types of phishing attacks: spear phishing, phishing email campaigns, vishing (voice-based phishing through telephone calls), and clone phishing (replication of a previously legitimate email with malicious links substituted) [3]. However, the most relevant form of phishing attack is URL phishing. Attackers of this type of phishing create fake hyperlinks that look like the real ones and then distribute them via social networking sites, email, web browsers, ... According to the Anti-Phishing Working Group (APWG), in the fourth quarter of 2025, there were 853,244 scams through unique phishing websites [1]. Big corporations such as PayPal, Facebook, Microsoft, Apple, Amazon, or banks frequently take risks. Dhamija et al. conducted landmark user studies demonstrating that 90% of users could be deceived if they encountered well-designed fake websites [4]. This incident shows that the market needs systems that can detect fake URLs and websites and can detect them on various platforms.

2.2. Cross-platform system

2.2.1. Web plugin

Browser extensions are an effective solution to detect phishing because they can analyze URLs in real time when a user clicks on a suspicious page. Plugin tools combine lexical URL characteristics, blacklist checks, and website content analysis to identify suspicious websites and display immediate warnings when a link matches phishing patterns. Therefore, regardless of whether the user clicks the link in the email or on social media or searches on the search bar and presses enter, it immediately intercepts navigation events and then analyzes the URL and the site content that the user wants to access. However, browser plugins only protect web browsers on laptops or PCs, so other platforms cannot be protected. That is the reason why mobile detection apps exist.

2.2.2. Mobile app detection

Mobile apps can only detect URLs and phishing websites by copying and pasting the URL into the app scan bar instead of intercepting a link on social media apps and analyzing it.

The main reason for this is that on mobile, each application runs in a restricted access permission environment, which does not allow other applications to intercept. Apple requires apps to follow the sandboxing mechanism. So every app used on an iPhone only allows access to resources and data within their granted permissions to protect data and prevent tracking or interference from other apps [14]. Similarly, Android requires apps to declare and request permissions before accessing sensitive data and only requesting permissions when absolutely necessary [15].

Therefore, a normal mobile app cannot freely block or analyze every link when a user clicks on other applications. If performing track links from apps will be considered data collection outside the app's core functionality, it violates the Google Play User Data policy [16].

2.3. URL Heuristic Detection Methods

Analyzing URL structure and length is one of the most common methods for detecting phishing. The detection system analyzes length, similar characters, and IP addresses or domains inside the URLs by checking heuristics and lexicals. If an IP address is used as a hostname, this is a sign of a phishing URL since organizations always use registered domains, not display IP addresses in URLs. When people see a long URL, which is exceeding 75 characters, it often hides the true destination by embedding the destination within a long parameter string [5]. Besides, when people see links or subdomains that include some suspicious keywords like "verify," "signin," "signup," "credentials," and "account," they should consider them suspicious because attackers are trying to convey the legitimacy of the URL through words related to personal account management. Additionally, if a company's, bank's, or brand's URL has multiple subdomains (e.g., login.verify.paypal.com), this is also a sign of a phishing URL. This technique uses the name of a reputable company to deceive ordinary users; however, the registered domain actually belongs to the attacker [6]. People can verify a safe link by checking the HTTPS protocol inside the URLs. If users see any sites using the HTTP protocol, they should consider it before clicking to continue because these sites may not be safe. Normally, companies barely use hyphens inside their domain name, so if a link includes hyphens between domain names, it is also a sign of phishing. (e.g., paypal-secure-login.com). Additionally, attackers also target URLs ending with executable file extensions or archive files (.exe, .apk, .zip, .rar) to distribute malware [7]. Studies have

shown that a detection system only having those rules can detect correctly up to 93% [8]. However, nowadays, attackers develop fake websites and URLs that can bypass these regulations. For example, they know that long URLs are suspected of being unsafe, so they use URL shortening tools to create short, fraudulent links. Therefore, detection systems nowadays not only need these rules but also need to apply machine learning, LLMs, semantic analysis, and logo analysis to verify websites correctly.

2.4. Threat Intelligence and Domain Analysis

Besides analyzing the structural characteristics of each URL, the system uses threat intelligence, WHOIS data, and SSL certificates to detect phishing.

Threat intelligence is analyzed data, which helps us to predict and prevent cyberattacks. Threat intelligence consists of blacklisted domain names and IP addresses. Blacklist is the simplest threat intelligence approach. It is a database containing IP addresses or domain names that have been identified as malicious or fraudulent. The common sources such as PhishTank, OpenPhish, and Google Safe Browsing allow real-time URL checking based on blacklists. This method is highly accurate for detected phishing sites, but a major limitation is the update delay. Many phishing websites only exist for a very short time and may disappear before being added into blacklisted [9]. Therefore, blacklists need to be combined with methods for analyzing URL behavior and characteristics.

In addition, to verify if a website is safe or not, the system also checks domain age from WHOIS data. Safe websites often use domain names for many years, while phishing sites often register new domain names to perform attacks. A domain that is under 30 days old has a high risk level, a domain that has an age from 30 to 90 days has a medium risk level, and a

domain up to 180 days old has a low risk level. However, checking a domain's age has limitations. Websites of large corporations hide their WHOIS information, so the system cannot check domain age. In order to avoid detecting legitimate domains with privacy protection enabled, the system only classifies them as small risk levels.

Finally, the system check of the SSL certificate to assess trustworthiness at the domain level. Since attackers can easily create free TLS certificates, nowadays, the HTTPS protocol is no longer a sure sign of a secure website. However, if a site misses a valid SSL certificate, a self-signed certificate, or an expired certificate, it is still a suspicious site. Therefore, checking the SSL certificate is a quick, low-cost, and useful step to add to a phishing risk scoring system.

2.5. Machine learning for phishing detection

Applying machine learning (ML) to phishing detection systems is a widely researched topic. The goal of this study is to address the limitations of traditional detection methods. The process of applying machine learning is that the system collects websites and URLs to create datasets, then learns and derives patterns to distinguish between safe and malicious links. This capability allows the system to automatically identify even new phishing schemes that blacklists have not updated yet.

To implement machine learning, the system must perform preprocessing and feature engineering. In the preprocessing step, the system processes raw data, removes missing data, and encodes categorical variables. After preprocessing, the system performs feature engineering. The main goal of this process is providing URL features to the machine learning model, which helps the model improve learning capabilities and enhances prediction

performance. Feature engineering transforms a raw URL into features such as URL length, number of dots or hyphens, IP address, or the HTTPS protocol [10]. These features not only highlight hidden patterns inside a URL but also help the model to distinguish between legitimate and fraudulent websites.

Sahingoz et al. conducted a study which compared several different machine learning models for phishing detection. This study used a dataset consisting of 73,000 URLs and implemented models such as random forest, neural networks, Naive Bayes, and support vector machines. The final insight which can be drawn from this study was that Random Forest and Neural Network had the highest accuracy, which was over 97%. This result made them become the two models that achieved the best results [11]. Currently, the XGBoost model is also widely used for phishing detection because it can improve accuracy by continuously correcting errors from previous models. Another important machine learning implementation is a threshold. The system evaluates a dataset by using metrics such as precision, recall, and F1-scores for thresholds from 0 to 100 points. After that, the system compares the results of all models to find out the best models and deploys them.

The important application of machine learning is the threshold. After performing machine learning, it evaluates the dataset using metrics such as precision, recall, and F1-scores and then calibrates the thresholds from 10 to 100 points. Finally, compare all the metrics results to find out the best models for deployment.

2.6. LLM

The phishing detection system applies large language models (LLMs) to process images. It solved the limitations in semantics and visuals of traditional detection systems. LLMs

analyzed full webpage screenshots, website content, HTML, and logos to detect brand impersonation and content fraud.

2.6.1. Logo detection

Researchers use models like Faster R-CNN, which are object detection models, to crop logos from websites and then request an LLM to identify the brand. The LLM uses models from Gemini or GPT to evaluate the logo accuracy. The accuracy result was around 82% to 83% [12]. However, if the system only relies on logos to identify the phishing websites, it often misses some sophisticated fraud websites. Studies show that detecting a fraudulent website by logo is less accurate, around 8% to 11%, than analyzing the whole website screenshot to detect it [12]. This is because large language models (LLMs) are prone to confusion between color and style characteristics [12]. For example, they might misidentify the Facebook logo and Coursera logo due to similarities in dominant color tones.

2.6.2. Content detection

Identifying phishing websites based on content has low accuracy. It varies from 54% to 63% [13]. The reason why it has low accuracy is that attackers can easily copy the whole UI and content of legitimate websites. To overcome this, the system should combine screenshots of the entire screen with the URL. It allows LMM to verify whether the image content actually matches the domain name, and the accuracy of the identification results can reach up to 94% [13].

2.6.3. Meta data and content analysis

Besides logo and content analysis, the system also needs to check HTML, page titles, and metadata tags because many phishing sites often impersonate brands by using almost the same titles, descriptions, and content as the official ones.

LLMs analyze HTML to identify several suspicious signs, such as unusual login requests, urgent messages, fake warnings, or content that does not match the real domain. Some studies show that this approach can achieve high accuracy in phishing detection. However, HTML is long and contains many elements such as CSS, JavaScript, menus, advertisements, or interface formatting code; it causes the model to exceed its token limits, and so the system cannot continue to analyze. Therefore, before sending data to the LLM, the system needs to clean and simplify the HTML by removing CSS, scripts, and unimportant elements, only keeping <meta> tags, main headings, important text content, and login forms. This step helps the model focus on analyzing content and then provides the website risk level more accurately.

2.7. Scoring system

The total risk score in the system is calculated by this formula:

$$\text{TotalRisk} = (\text{RulesScore} * \text{rules_weight}) + (\text{ml_probability} * 100 * \text{ml_weight}) + \text{LLM_cross_match_score} + \text{PenaltyPoints}$$

This formula is taken based on the fundamental equation that was established by NIST 800-30 and ISO/IEC 27005. [25]

The formula: Risk = Likelihood * Impact [25]

Likelihood: probability of a threat occurring

Impact: the magnitude of harm if it occurs.

In the scoring system

$$\text{Likelihood} = (\text{RulesScore} * \text{rules_weight}) + (\text{ml_probability} * 100 * \text{ml_weight})$$

RulesScore: Represents known risk patterns

.ML Probability: Represents predictive uncertainty based on statistical models.

Weights: Used to balance the reliability of the rule engine versus the machine learning model.

$$\text{Impact} = \text{LLM_cross_match_score} + \text{Suspicious penalties}$$

CHAPTER III

Methodology

3.1. System Overview and Architecture

3.1.1. System overview

The product provided throughout this research is a hybrid phishing detection system, which performs effectively in real-time and real-world scenarios. The main objective of this system is to create a multi-layer detection architecture that combines three main approaches: a rules-based method, training machine learning models to evaluate threshold and scoring systems, and applying large language models for advanced analysis. Therefore, the system not only detects suspicious signs in the URL's structure but also analyzes the content, logo, and metadata of the website.

There are two platforms that support this system. The first platform is a web plugin. Users load the extension into many web browsers, such as Google, Microsoft Edge, and Firefox, to use the system. The second platform is a mobile app. The next support platform is a mobile app, which users can use on both Android and iOS. Both platforms act as user-side interfaces, allowing suspicious URLs to be sent to a backend system for analysis and to receive security alerts.

3.1.2. Architecture

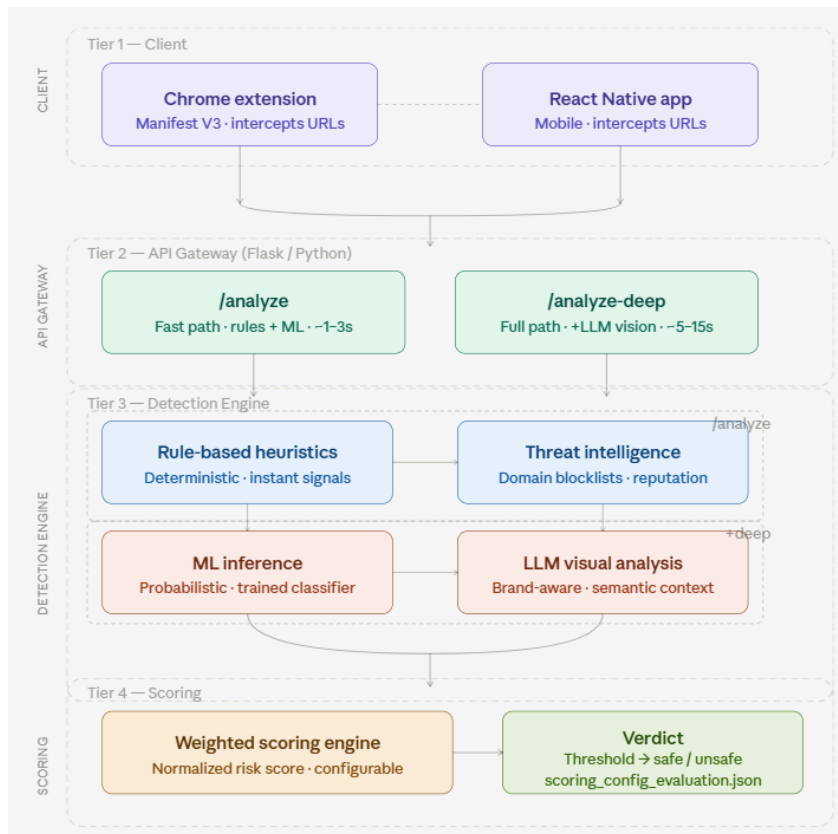


Figure 3.1.1: architecture of phishing detection system

The system has two client platforms: a web browser extension and a mobile application. Both of them are user interfaces and perform as tools to collect or intercept URLs and send them to a backend for analysis. The backend is designed as a REST API server, which shares logic through multiple platforms.

The architecture is designed into four main functional tiers: the client tier, API gateway tier, detection engine tier, and scoring tier.

The first tier is the client tier. This tier consists of the extension and the mobile app.

Technology to develop client tiers is Manifest V3 for plugins and React Native for mobile apps. In both platforms, users can enter suspicious URLs in the search bar and check them.

The difference between the web version and the app version is about the interception

mechanism. In the browser extension, URLs can be intercepted directly everywhere that they want, but in the mobile app, users are only allowed to paste URLs manually.

The second tier is API Gateway. It is responsible for receiving request URLs and sending them to the appropriate backend. The backend provides two endpoints: /analyze and /analyze-deep. The /analyze endpoint performs a fast check for links. It only analyzes links through rule-based methods and machine learning. Therefore, users only need to wait from 1 to 3 seconds to get the final results. Meanwhile, the /analyze-deep endpoint performs checking in more detail. Besides checking rule-based and machine learning, it also checks content, images, and logos by using LLMs. However, users need to wait a longer time to get a response since the result depends on page load speed and model response time.

The third tier is the detection engine tier, which includes the core detection logic. This tier is divided into four small sub-modules. The first module is rule-based heuristic checks. This module checks URL structure to verify if URLs contain suspicious keywords, have wrong domain names, have invalid URL lengths, consist of IP addresses inside the domain name, or do not use the HTTPS protocol. If it violates one of these, it will add to a score that is considered a phishing site. The second module is the threat intelligence and domain checking module. This module checks domain reputation, WHOIS-related information, domain age, and blacklist indicators. The third module is machine learning. This module applies trained classifiers to extract probabilistic patterns from URL and webpage features. The last module is LLM. It will analyze the page content, logo identity, and metadata in order to detect sophisticated fake websites that traditional rules cannot catch.

The final layer is the scoring layer. This score is the take from scores that each module above obtains and then applies to a specific formula to calculate the total risk score. The overall risk score will be compared with the threshold to determine whether the URL is safe or unsafe.

3.2. Multi-layer phishing detection approach

3.2.1. Rationale for Hybrid Detection

Uses a hybrid model with these checks:

- Rule-based detection: use for fast, understandable checks.
- Threat intelligence: use for recognizing malicious domains.
- Domain/SSL checks: used for checking the domain reputation and certificate.

3.2.2. Rule-Based Heuristic Detection (Layer 1)

Rule-based heuristics are the first detection layer, giving penalty points for each rule when URLs are violated. When URL violation rules are made, their scores are accumulated.

Rule 1: The hostname uses an IP address instead of displaying a domain.

If a website has its hostname IPv4 or IPv6 address instead of domain name, it will gain a penalty of 80 points. This is because real websites use a registered domain name for users to easily remember and access instead of using IP addresses. So if a website consists of an IP address inside the URL, it means that it is an adhoc server, so it may not be a safe website to access.

Rule 2: URL length

The URL with length more than 100 characters gets 20 points. A long URL is considered to be a sign of a phishing link because attackers built long URLs to hide the true destination through lengthy parameter strings and redirect chains.

Rule 3: suspicious keywords

Here are group of common suspicious keyword:

- Security, Account Action:

login, signin, verify, account, security, update, secure, confirm, alert, suspended, unusual, authentication, restore, unlock. [17]

- Financial, Banking:

bank, pay, payment, billing, invoice, refund, transaction, wire, funds, crypto, bitcoin. [17]

- Brand Impersonation:

amazon, apple, microsoft, google, facebook, netflix, paypal, dropbox, fedex, dhl, chase, wells Fargo. [17]

- Urgency/Deception:

free, win, urgent, limited, winner, reward, gift, bonus, claim [17]

If the URL consists of keywords from these suspicious groups, it will get 15 points. The reason why URLs include these keywords is considered to be unsafe because these terms are systematically embedded by phishing operators to simulate legitimate account management contexts.

Rule 4: Non-HTTPS schemes

When a user checks a link with HTTP protocol, the checking system penalizes it with 30 points. Even though the lack of HTTPS protocol may not be strong evidence to show that it is a phishing link, it still shows that this link is not secure since this link does not have SSL/TLS encryption certificates to protect data exchanged between the browser and the server.

Therefore, attackers can steal information more easily.

Rule 5: Excessive Subdomains

Hostnames that include more than 3 dots or include four or more sub domains in the right side of URL get a 25 points penalty. Normally, users read URLs from left to right, they habitually

trust and access the sites when they meet domains that come from large corporations, banking,... However, in reality, the actual registered domain name is the last two domains and this belongs to the attacker.

Rule 6: Suspicious File Extensions

If a user clicks a hypertext link that ends with these file extensions “.exe, .rar, .zip, .apk,...” are penalized 40 points because attackers can compress malware inside this.

Rule 7: Multiple Dashes in Domain Name

Domain names that include more than two hyphen characters get 15 points because this structure often combines brand names with modifier words, which causes users to believe this is the official website.

3.2.3. Threat Intelligence (Layer 2)

There are some existing sources, such as PhishTank, OpenPhish, Google Safe Browsing, that store phishing links from the Internet. When users check a link, this link also needs to be checked from these sources. If the link they want to access is found in the source, the system immediately classifies it as unsafe and applies a strong penalty score (100 points)

3.2.4. Domain and SSL Analysis (Layer 3)

In the domain and SSL analysis layer, the backend analyzes two things, the first one is WHOIS and the second one is SSL certification validation.

- WHOIS domain age analysis

The backend query queries WHOIS data to get domain registration information then stripping sub domain and only keeping the register domain. After that, the system compares the registration date with the current date to verify how long the domain works.

Table 3.2.1: Domain age table with risk level and its penalty point

Domain Age	Risk Level	Penalty
< 30 days	High Risk	40 points
30–89 days	Medium Risk	20 points
90–179 days	Low Risk	10 points
Hidden WHOIS / unretrievable	Uncertain	8 points
≥ 180 days	No penalty	0 points

The reason why we need to analyze domain age is because attacks often use "disposable" domains, which are registered just hours or days before the attack begins. However, big brands often hide their WHOIS to protect competitive intelligence, privacy, new plans and strategy [18] so attackers may take advantage of this point to hide their WHOIS data.

- SSL certificate validation

When the backend receives URLs consisting of HTTPS protocol, it creates a Transport Layer Security (TLS) connection to the server via port 443. Next, the server takes their digital certificate and verifies it with the system's trust certificate [19]. SSL is considered to be

unsafe if it is an invalid SSL, expired SSL and Self-signed SSL. So when the HTTPS has the SSL result, they get 35 penalty points.

3.3. Machine Learning Model Development

3.3.1. Prepared dataset

The system uses the dataset “Web page phishing detection dataset [20][21]” to do the machine learning. The provided dataset includes 11430 URLs with 87 extracted features related to structure and syntax of URLs, content page and external service queries [21]. Phishing and legit pages are label 1 and 0.

3.3.2. Preprocessing and feature engineering

3.3.2.1 Feature engineering

Feature engineering is a process to extract and select important attributes from raw dataset [22], which are raw URLs to help machine learning models easily distinguish between legit and phishing websites.

This dataset is divided into 6 categories:

- Lexical features:

This category focuses on syntax, length, host name, special characters such as dot, hyphens, slash, question mark inside raw URL.

- Structural features

This category focuses on sub domains and abnormal sign of URLs. It extracts the number of sub domains inside a URL and verifies if the raw URL consists of IP address, port or shortening services.

- Statistical features

This feature calculates the ratio of digit to character, digit host. Besides, it also statistics shortest average characters and longest average characters in URL.

- Semantic features

This feature is used for finding suspicious keywords in a common suspicious group or checking if a well-known brand is shown in a sub domain.

- Content features

Analyze content of websites such as login form, iframe, popup,...

- External service features

Use some external services to verify such as WHOIS to check domain age, web traffic to track number of access,...

3.3.2.2. Preprocessing

- Handle missing value

Place 0 for null values and median for ratio features.

- Class balance verification

In phishing detection, the sample of safe and phishing must be equal. This ensures the model does not have any bias. To perform this, use oversampling and undersampling technique

- Feature scaling

Each URL has a different structure, length but the number of dots inside each link only varies from 1 to 5. To perform this step, use StandardScaler to transform data so the mean is 0 and bias is 1. This technique helps algorithms to avoid being distributed by characteristics with larger numbers.

- Train-test split

Divide the dataset into 2 parts, the training part takes 80% of data, testing part takes 20% of data.

3.3.3. Model selection and training

Use five supervised classification algorithms to train and evaluate the dataset. After that, find out which model is the best for deployment.

- Decision tree

This is the simplest model. It is a single interpretable tree classifier.

This model is suitable for becoming a baseline to view the benefit of other models.

- Random forest

This model is a collection of decision trees. It is suitable to use in a detection system because to make a final prediction, it must sum up results from multiple trees. It offers a strong tabular data performance and minimizes the errors.

- XGBoost

A gradient-boosted tree ensemble that iteratively minimizes classification error by training each new tree on the residuals of previous trees.

- Support vector machine (SVM)

Use the Radial Basis Function (RBF) kernel to separate data into multi-dimensional space. It found out the boundary between the safe and unsafe sites.

- Multi-layer perception (MLP) - Neural network

A feed-forward neural network with one or two hidden layers, representing the deep learning baseline.

Use scikit-learn to train models and XGBoost to train its own models.

3.3.4. Model evaluation

After train each model on 80% data, use 20% left data to test and evaluate by these metrics: accuracy, precision, recall, F1-score, ROC-AUC

Key terms of confusion matrix:

TP (True Positive): Predicted positive, actually positive. [23]

TN (True Negative): Predicted negative, actually negative. [23]

FP (False Positive): Predicted positive, actually negative (Type I error).[23]

FN (False Negative): Predicted negative, actually positive (Type II error) [23]

- Accuracy:

Accuracy shows the ratio of predicted instances to whole instances in the dataset. It provides an overview about the performance of each model.

Formula:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

- Precision

When URLs are predicted as phishing, precision metrics help users to find out the proportion of real phishing instances over all URLs that are predicted as phishing. This metrics minimize the false alarms on

Formula:

$$\text{Precision} = TP / (TP + FP)$$

- Recall

Recall helps to identify the proportion of phishing URLs that are correctly identified in the dataset. This metric minimizes the missed caught case during detection.

Formula:

$$\text{Recall} = TP / (TP + FN)$$

- F1-score

F1-score is a harmonic means of precision and recall, used for evaluation models performance when it needs to balance precision and recall metrics. The higher F1-score, the higher score in precision and recall metrics. Therefore, high F1-score indicates that both the false positive errors and false negative errors are reduced.

Formula:

$$\text{F1-score} = 2 * ((\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}))$$

- ROC-AUC

ROC curve visualizes the metric performance. This performance is the area under the curve (AUC) that is created by plot True Positive Rate (TPR) against the False Positive Rate (FPR)

AUC (Area under the curve)

The AUC measures the capability to distinguish between phishing URL and legitimate URL at various decision thresholds.

Formula:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d\text{FPR}$$

3.4. Large language models (LLM)

All of the previous layer is only analyzed URL structure, length, domain and verify if it violates rule or not. This LLM layer is an advanced section. The system uses the Google Gemini 1.0 Flash or Google Gemini 2.0 Flash model to perform semantic analysis of brands.

3.4.1. Page content retrieval

The first step to do LLM was to take the target raw URL. The target raw URL was retrieved via an HTTP GET request in `fetch_page()` function

Algorithm 1 `fetch_page`: Webpage Content Retrieval

Input: `url`, `timeout` (default 10)

Output: Tuple (`html_text`, `final_url`)

```

1: try
2:   response  $\leftarrow$  GET request to url with timeout and verify = False
3:   if response.status is error then raise Exception
4:   return (response.text, response.url)
5: except Any Error do
6:   return (None, url)
7: end try

```

Figure 3.4.1: `fetch_page()` function

In order to optimize the accessibility and avoid server defenses, the system emulates a User-Agent that looks like the standard desktop browsers. Then the system tracks and processes all the redirect chains to determine the final destination address. This action exposes the sophisticated landing page that attackers apply masking techniques to hide.

3.4.2. Logos and Favicon analysis

The goal of this analysis is to verify if any websites use the trademark of the organization illegally. The system search logo by performing these steps:

Step 1: Scan the `<link rel="icon">` tag or shortcut icon. Then check the Open Graph meta tag (`og:image`).

Step 2: Look for `<img>` tags within containers like `<header>` or `<nav>` that contain the keyword "logo".

Step 3: Check the default path `/favicon.ico`.

Algorithm 2 `extract_logo_url`: Visual Identity Extraction

Input: `html`, `base_url`

```

1: soup  $\leftarrow$  Parse html; base  $\leftarrow$  Hostname of base_url
2: Check <link rel="icon">  $\rightarrow$  if found return Absolute URL
3: Check Open Graph (og:image)  $\rightarrow$  if found return Absolute URL
4: Heuristic: Search <img> tags with "logo" in attributes within header/nav
5: Fallback: return base + "/favicon.ico"

```

Figure 3.4.2: `extract logo from url` function

The extract logo is downloaded and transferred to Gemini Vision models. To analyze the logo, Gemini performs this prompt "Analyze this logo/favicon image carefully. Which company or brand does this logo belong to? Does the logo look official, or modified/distorted/fake? Rate authenticity 0–100 (100 = definitely official). Respond in this JSON format: {"brand": "CompanyName or unknown", "is_official": true/false, "authenticity_score": 0-100, "analysis": "Brief explanation"}"

The JSON response is parsed to extract: logo_brand (string), logo_confidence (0–100), and logo_is_suspicious (boolean).

Algorithm 5 analyze_logo: Visual Brand Identification

Input: *logo_url*

```

1: result  $\leftarrow$  Initialize default unknown values
2: try
3: Download img from logo_url; Convert to RGBA
4: prompt  $\leftarrow$  "Identify brand and authenticity score (0-100)"
5: res  $\leftarrow$  CALL VisionModel.generate(prompt, img)
6: return ParseJSON(res)
7: except return result with error
  
```

Figure 3.4.3: analyze logo function

3.4.3. Page content analysis

The LLM read the text content and structure of the page to determine the page's objective to clarify if this is a login page, a payment page, or a news page, etc and detect if sites consist of any signs, encryption, or requests for sensitive information

Content is extracted from the page HTML using BeautifulSoup4. Form elements and their input field types are also extracted to detect credential-harvesting login forms.

Algorithm 4 `extract_page_content`: Content Sanitization

Input: *html*

- 1: *soup* $\leftarrow$ Parse *html*; Remove $\{script, style, noscript\}$
 - 2: *parts* $\leftarrow \emptyset$
 - 3: **Extract Forms:** Store action URLs and input types/placeholders
 - 4: **Extract Headings:** Store text from $\{h1, h2, h3\}$ (max 10)
 - 5: **Extract Body:** Store first 1500 characters of visible text
 - 6: **return** Join(*parts*)
-

Figure 3.4.4: extract page content function

The extracted content is sent to the Gemini model with a prompt requesting: "What type of website is this? Which brand does this page claim to be? Does the content appear legitimate? List suspicious indicators."

Algorithm 6 `analyze_content`: Semantic Text Analysis

Input: *page_content, url*

- 1: *prompt* $\leftarrow$ "Analyze content from *url*: identify brand, type, and legitimacy"
 - 2: *res* $\leftarrow$ **CALL** TextModel.generate(*prompt*)
 - 3: **return** ExtractJSON(*res*)
-

Figure 3.4.5: analyze content function

The result returns are type, brand, content_is_suspicious and reason.

3.4.4 Meta data analysis

The system focuses on collecting HTML tags that are placed inside the <head> part. It extracts data from these tags <title>, <meta name = "description">, <meta property="og:title"> and <meta property="og:site_name">.

Algorithm 3 `extract_metadata`: Meta Tag Analysis

Input: *html*

Output: Dictionary of title, description, and Open Graph tags

- 1: *meta_dict* $\leftarrow \emptyset$; *soup* $\leftarrow$ Parse *html*
 - 2: *meta_dict.title* $\leftarrow$ *soup.title*
 - 3: **for all** meta tags in *soup* **do**
 - 4: Extract 'description', 'og:title', 'og:site_name'
 - 5: **end for**
 - 6: Remove $\{script, style, noscript\}$; *meta_dict.body* $\leftarrow$ *soup.text*[0 : 2000]
 - 7: **return** *meta_dict*
-

Figure 3.4.6: extract metadata function

After extracting, send the data to Gemini models to claim identity through title, description, website of which organization. Besides identifying identity, it also checks for mismatches. The LLM compares the "self-proclaimed identity" in the metadata and the "actual identity" of the website via URL or other elements to verify if there exists any discrepancy or not.

3.4.5. Cross-matching

The system uses a `cross_match` method to synthesize all the results from logo and favicon analysis, content analysis and meta data analysis. The purpose of this action is to verify whether the identified brands match perfectly with the existing domain name (URL).

The system collects a list of brand domains from big corporations, banks, etc that usually are impersonated

If users access corporations or banks' websites but when checking this URL, website trade mark and content that do not match with the real domain that is stored inside the list, it will get a penalty score like cross-match penalty table.

Table 3.4.1: cross-match penalty

Condition	Penalty
Logo brand does not match domain	+15 points
Content brand does not match domain	+25 points
Both logo AND content mismatch (neither unknown)	Additional +40 points
Metadata mismatch detected	+10 points

3.5. Weighted Scoring Engine and Threshold Calibration

3.5.1. Score aggregation formula

$$\text{TotalRisk} = (\text{RulesScore} * \text{rules_weight}) + (\text{ml_probability} * 100 * \text{ml_weight}) + \text{LLM_cross_match_score} + \text{PenaltyPoints}$$

Explain:

RulesScore: Scores come from traditional rules (such as blacklists, SSL certificate checks).

ML Probability: Probability (from 0 to 1) from a Machine Learning model, multiplied by 100 to a 100-point scale.

LLM Cross-match Score: Penalty points from the cross-matching process (explained in section 3.4.5).

Penalty Points:

+10 points if logo_is_suspicious (Logo is suspected of being fake).

+15 points if content_is_suspicious (Content is deemed suspicious by Gemini).

3.5.2. Threshold calibration

The threshold is calibrated through the evaluation process.

The full scoring pipeline is applied to the test dataset, simulating realistic deployment conditions. For each threshold, the system calculated:

- True Positives (TP): Phishing URLs correctly classified as unsafe
- False Positives (FP): Legitimate URLs incorrectly classified as unsafe
- True Negatives (TN): Legitimate URLs correctly classified as safe
- False Negatives (FN): Phishing URLs incorrectly classified as safe

After getting the result, the system defined the final threshold.

3.6. Tool and technology

3.6.1. Browser Extension (Desktop)

WebExtension API (Manifest V3): used for intercept requests that access browsers, manage the permissions for accessing the website, and control tab behavior.

JavaScript (ES6+): Implements URL interception logic, backend communication, and message passing.

HTML, CSS: Used for building the UI when meeting the URL that is phishing.

Chrome Extension APIs:

- **Webrequest:** intercepts navigation events
- **Storage.local:** stores trusted (whitelisted) domains
- **Tab and runtime** redirects and inter-component messaging

3.6.2. Backend Service

Language: Use Python to code for the URL analysis and scoring logic.

Framework: use Flask since Flask is a lightweight web framework exposing the amylase of REST API.

Communication protocol: HTTP + JSON

This is a communication protocol between extension and mobile client and backend.

3.6.3. Machine Learning

Classification Algorithms: Decision Tree, Random Forest, XGBoost, SVM, MLP

Evaluation Metrics: Accuracy, Precision, Recall, F1-score.

3.6.4. Mobile Application

React native application

3.6.5. Development Tools

Browser Developer Tools (Chrome DevTools)

Git (Version Control)

CHAPTER 4

IMPLEMENT AND RESULTS

This chapter introduces the core technology and development process. The hybrid phishing detection system has four main phase:

- Backend API development
- Performing model training and evaluation
- Refine threshold and scoring tool
- Deploy application on client platform

4.1 Backend API Server Implementation

The backend is implemented as a single file application using Flask. In this file, it consists of four RESTful endpoints. In order to share resources globally, the backend uses Cross-Origin Resource Sharing (CORS) through flask-cors. Therefore, users can make requests from multiple platforms such as the browser and mobile applications.

4.1.1. Run the backend

Open the terminal and type the directory that stores the backend file to start the server.


```
PS D:\Phishing-Detection-Web-Tool\backend> py app.py
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
[ML] Model loaded: XGBoost (F1=0.9677980852915579)
[ML] Features: 98 columns
ML Model: ☒ Loaded
LLM: ☒ Available
Threshold: 18
Weights: {'rules_weight': 1.0, 'ml_weight': 0.3, 'content_mismatch_penalty': 25, 'logo_mismatch_penalty': 15, 'cross_match_penalty': 40}
=====
Endpoints:
POST /analyze - Fast (Rules + ML)
POST /analyze-deep - Deep (Rules + ML + LLM)
GET /config - View config
POST /config/reload - Reload config from disk
=====
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.8:5000
Press CTRL+C to quit
```

Figure 4.1.1: Running the backend server

4.1.2 API Endpoints

Table 4.1.1: API end point

Endpoint	Method	Description	Avg. Response Time
GET /	GET	Show the status of system and module is available to check	Smaller than 100 milliseconds
POST /analyze	POST	Fast analysis with rule-based an ML	1- 3 seconds
POST /analyze-deep	POST	Deep analysis: use Rules-based, ML, LLM	5 - 15 seconds

Endpoint	Method	Description	Avg. Response Time
GET /config	GET	View current scoring and threshold	Smaller than 100 milliseconds
POST /config/reload	POST	Hot-reload scoring config from disk	Smaller than 200 milliseconds

4.1.3 Request and Response Structure

When receiving an URL, the backend perform these steps:

Step 1: Check the structure of data. The structure must be in JSON format.

Step 2: Performing feature extraction by analyzing URLs into smaller components such as length, number of dots, occurrence of sensitive keywords, etc.

Step 3: Calculate scoring from heuristic check and machine learning.

Step 4: Response back safe or unsafe status along with a risk score and recommended action

Here is the example about response from an endpoint (at here it is the /analyze-deep endpoint) for a phishing URL

```
127.0.0.1 - - [05/May/2026 00:38:39] "POST /analyze-deep HTTP/1.1" 200 -
[LLM] Analysis complete. LLM Score: 0, Risk: low
-> RULES SCORE: 8.0
-> ML SCORE: 10.73 (prob=0.3575)
-> LLM SCORE: 0
-> TOTAL SCORE: 18.73
-> THRESHOLD: 18
-> STATUS: UNSAFE
-> LLM RISK: low
```

Figure 4.1.2: response from an endpoint for phishing URL

4.1.4 Module Loading and Initialization

When the server start running, the backend prints a diagnostic summary of each component:

```
[ML] Model loaded: XGBoost (F1=0.9677980852915579)
[ML] Features: 98 columns
ML Model: ☒ Loaded
LLM: ☒ Available
Threshold: 18
Weights: {'rules_weight': 1.0, 'ml_weight': 0.3, 'content_mismatch_penalty': 25, 'logo_mismatch_penalty': 15, 'cross_match_penalty': 40}
```

Figure 4.1.3: backend diagnostic summary

To ensure stability, the system applies a lazy loading technique at start time. The machine learning model is only actually loaded into memory when the receive the first URL check request. This helps the server start up faster. Besides, weight parameters are managed through the `scoring_config_evaluation.json` configuration file. If this file is missing or corrupted, a fail-safe mechanism will automatically activate, using the default values to maintain server operation, ensuring the system remains stable even under the bad conditions.

4.2. Machine learning models result

4.2.1. Models comparison

Five classifier algorithms were trained on the train set. After that, evaluate the test set. The table summarizes the performance of each model across all five evaluation metrics.

Table 4.2.1: Performance Comparison of Trained Classifiers

Model Name	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.9633	0.9592	0.9676	0.9634	0.9931
XGBoost	0.9676	0.9628	0.9729	0.9678	0.9948

Decision Tree	0.9423	0.9415	0.9431	0.9423	0.9533
SVM	0.9519	0.9559	0.9475	0.9517	0.9885
Neural Network (MLP)	0.9571	0.9595	0.9545	0.9570	0.9909

From the picture and table, XGBoost is the best model because it has the highest F1-Score.

Besides, other metrics of XGBoost also receive highest scores across compare to the other:

- **Accuracy (0.9676):** It correctly classifies the highest percentage of both phishing and legitimate URLs.
- **Precision (0.9628):** Among the URLs it predicts as phishing, a very high percentage are actually phishing.
- **Recall (0.9729):** It correctly identifies a very high percentage of actual phishing URLs.
- **F1-Score (0.9678):** This is a harmonic mean of Precision and Recall, providing a balanced measure. XGBoost has the highest F1-Score, indicating a strong balance between identifying phishing sites and avoiding false positives.
- **ROC-AUC (0.9948):** The Area Under the Receiver Operating Characteristic Curve is very close to 1, suggesting that the model has an excellent ability to distinguish between the phishing class and legitimate class.

4.2.2. Feature importance analysis

This analysis aims to determine which factor is the most important role to help Machine Learning models classify websites as safe or fraudulent among hundreds of factors. The experiment was taken through three models, which is XGBoost, Random forest and Decision

tree. The reason why choosing Random Forest and Decision Tree is because their accuracy, precision, recall is strong.

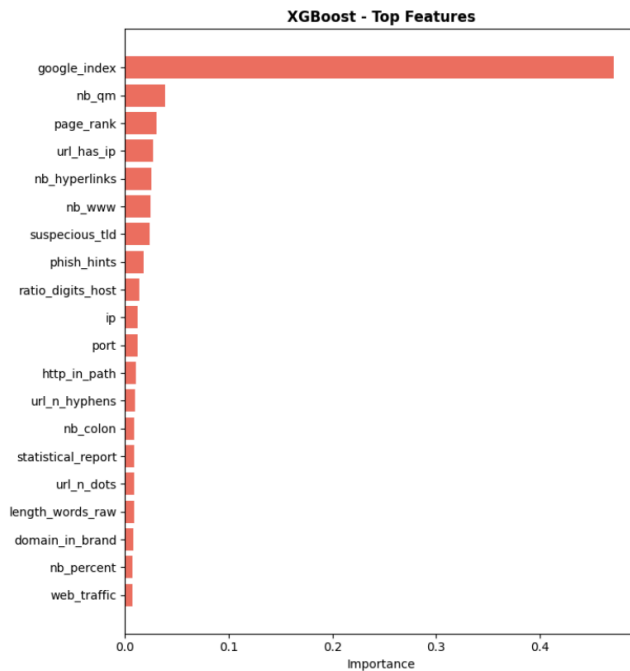


Figure 4.2.1: XGBoost top feature importance

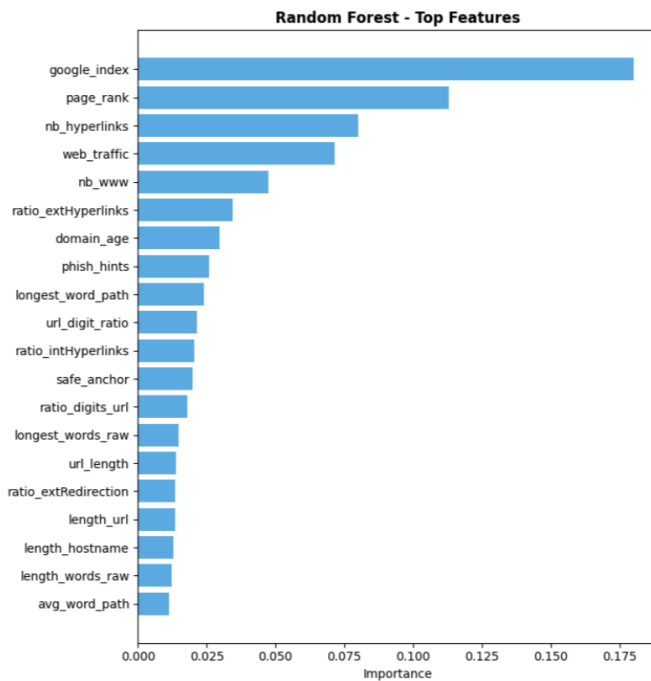


Figure 4.2.2: Random Forest top feature importance

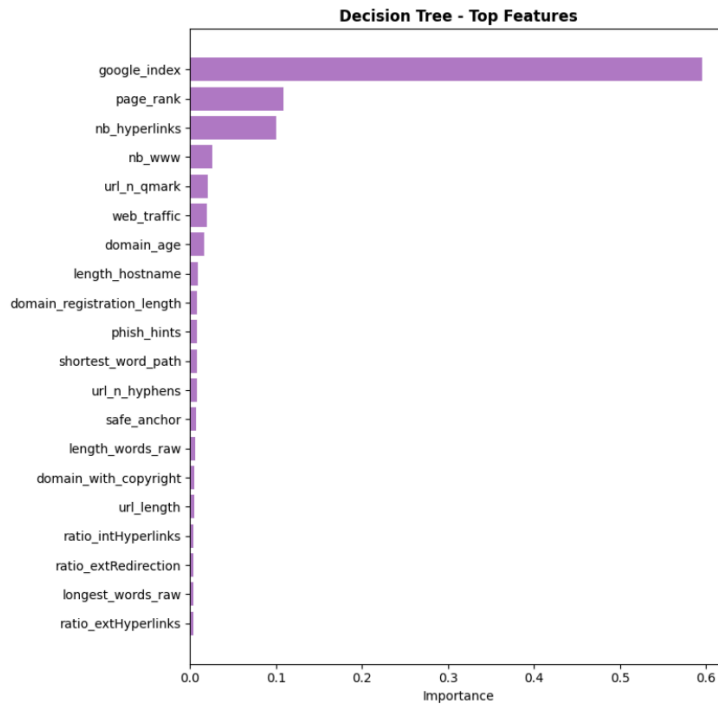


Figure 4.2.4: Decision tree top feature importance

Based on three visualization charts above, there are 5 features that provide the strongest distinguishing signals:

- Google_index:

This is the most important feature since it shows a list of websites that consist of indexes provided by Google. Normally, malicious domains rarely appear in Google's index, so this is the strongest indicator of fraud.

- Page_rank:

Phishing websites usually have a short lifespan so the PageRank number of these websites are less than official websites.

- Nb_hyperlinks (number of hyperlinks):

Phishing websites have less hyperlinks compared to official ones.

- Web_traffic and domain_age:

Similar to heuristic rule base, websites with low traffic and young domain ages are considered at phishing websites

- Nb_qm (number of question marks):

If URLs include too many question marks, these URLs may hide the real destination. So users may be vigilant with the URLs.

4.2.3 Model Serialization

The final XGBoost model combined with StandardScaler training and 100 input features are saved into a .pkl file by using joblib. When the backend starts, the model is loaded then it analyzes the URL with corresponding endpoints.

4.3 Threshold Calibration and Scoring System

4.3.1. Score Distribution Analysis

This section analyze how score risks level from 0 to 100

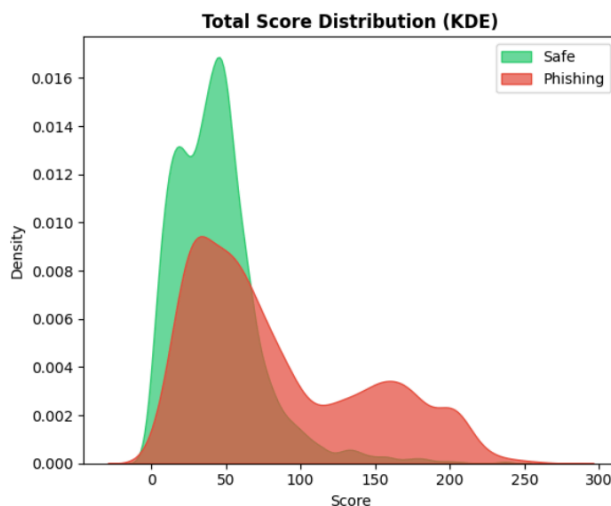


Figure 4.3.1: Total Score Distribution

The safe URL is distributed strongly between 0 and 50 points. It reaches a peak at 30 to 40 points. The phishing URL is distributed strongly in between 0 and 100. It reaches a peak at 10 to 20 points. There exists an overlap from score 20 to 80, which means that no single threshold perfectly separates the two classes.

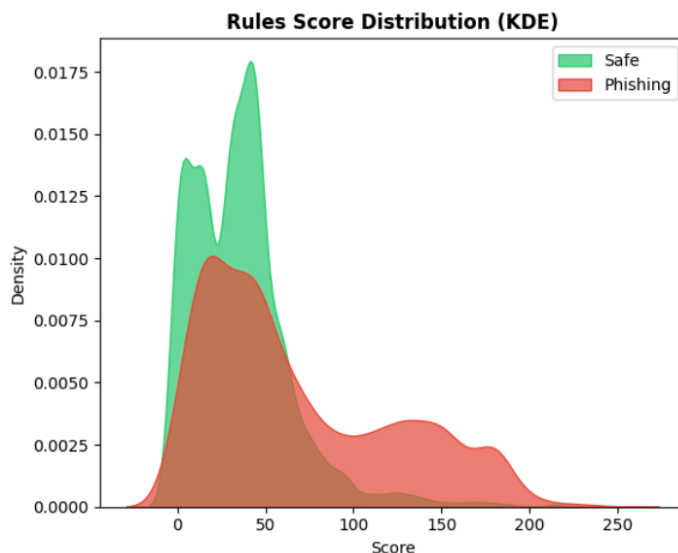


Figure 4.3.2: Rules Score Distribution

The Rules Score Distribution chart has a similar pattern to the Total Score Distribution, showing:

Safe URLs: Strongly distributed between 0 and 50 points, peaking most commonly between 30 and 40 points.

Phishing URLs: Distributed between 0 and 100 points, but peaking at a lower level, between 10 and 20 points.

Overlap Area: There is significant overlap between the scores of safe and phishing URLs, ranging from 20 to 80 points. This indicates that there is no single score threshold that can perfectly separate these two classes of URLs.

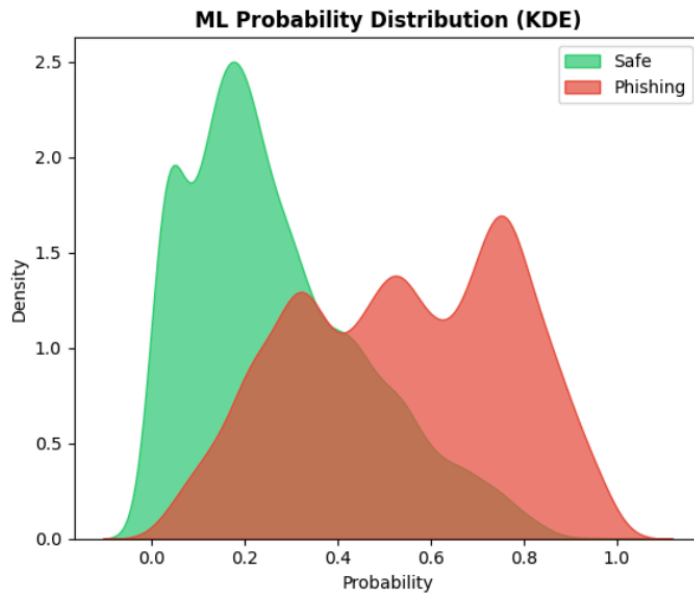


Figure 4.3.3: Machine Learning Probability Distribution

The probabilistic results from the ML model show a clear capability to distinguish between safe and phishing URLs. The majority of safe URLs have a low probability, it varies around 0.1 to 0.2. In contrast, the majority of phishing URLs have a much higher probability, concentrated in the range of 0.7 to 0.9.

Only a small fraction of phishing URLs fall into the hard to distinguish area. This area has probability around 0.2 to 0.4. Compared to the rule-based method, this overlap is significantly smaller. This indicates that the machine learning layer helps the system distinguish better than rule-based.

4.4.2 Confusion Matrix Analysis at Multiple Thresholds

In this part, use a threshold from 30 to 60 to compute confusion metrics and to illustrate the precision-recall trade-off. (View the detail at appendix figure A.1)

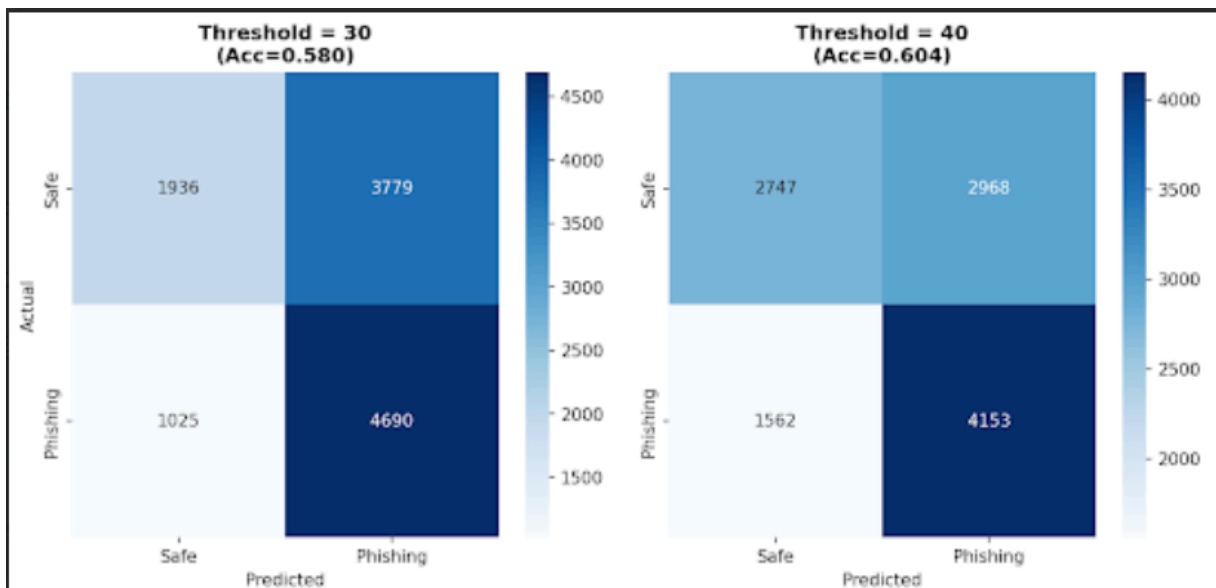


Figure 4.4.1: Confusion Matrices at Thresholds 30, 40

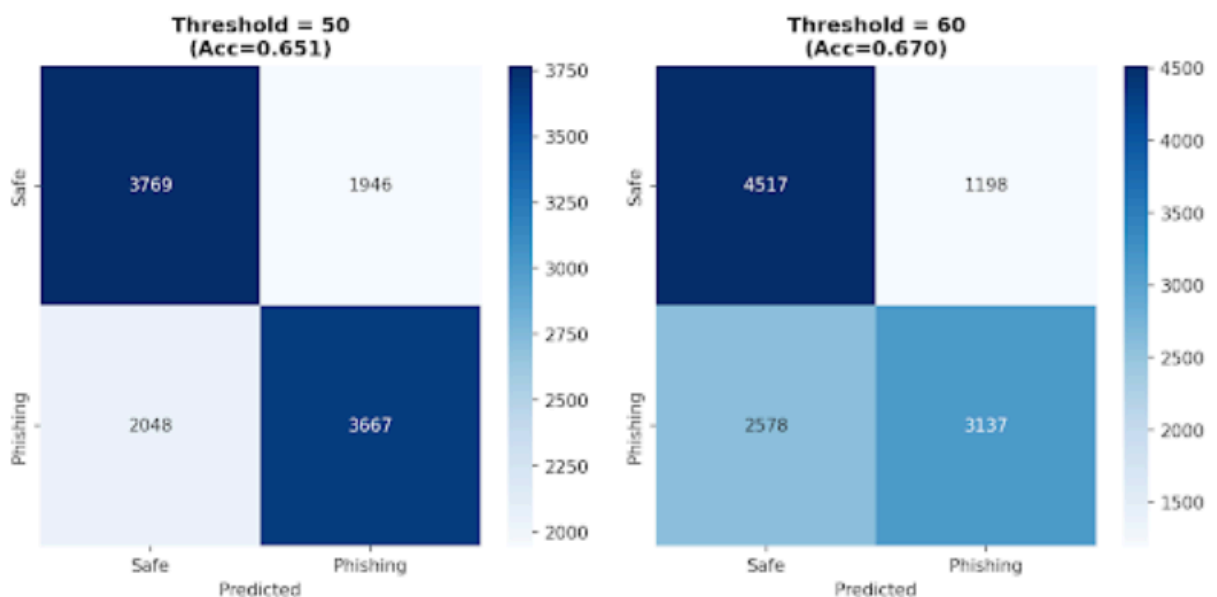


Figure 4.4.2: Confusion Matrices at Thresholds 50, 60

- Low Threshold (Threshold = 30) - Aggressive Scenario:

Advantages: Detected the maximum number of threats, capturing 4690 phishing URLs.

Disadvantages: Blocked 3779 safe websites by mistake.

Assessment: The false positive rate is too high, making it impractical for real-world implementation due to significant user disruption.

- High Threshold (Threshold = 60) - Conservative Scenario:

Advantages: Significantly reduced the false positive rate, affecting only 1,198 safe websites.

Disadvantages: Missed 2,578 phishing URLs.

Assessment: Weakens security commitments, exposing users to high risk.

- Optimal Threshold (Threshold = 45): helps the system achieve the necessary balance between safety and user experience.

4.5. LLM Analysis Result

The LLM analysis process was validated and categorized into four types: verified legitimate websites, typosquatting detection, brand-impersonation and maximum risks.

4.5.1. Legitimate websites

This group type has score = 0

Characteristics: Old domain age, the DNS resolves correctly, and there are no signs of brand impersonation, etc

Result: The system does not apply any penalties, allowing normal access.

4.5.2. Typosquatting detection

This technique is targeted to users who type the address incorrectly.

Characteristics: The domain name is similar to the real page. It is only different in 1 character and if the user reads quickly, they may skip it.

Result: Get penalties around 40 to 50 points

4.5.3. Brand-impersonation

This behavior is intentional use of a brand name in an unofficial domain.

Characteristics: The domain contains brand keywords but is not owned by that organization.

This type is often combined with technical errors such as DNS resolution errors.

Result: The penalty is 60. Since the score exceeds the safety threshold, the system identifies it as a fraudulent site and blocks it.

4.5.4. Maximum risk

These are blatant phishing websites that violate numerous security standards.

Characteristics:

Using an insecure protocol (HTTP).

Using unusual domain extensions (e.g., .xyz).

Subdomains that are too long and contain many hyphens.

Impersonating a major brand.

Result: The score is often double the blocking threshold, allowing the system to confidently assert that this is phishing with absolute certainty.

Table 4.5.1: LLM Analysis Test Results

URL	Verdict	Total Score	Primary Detection Signal
https://www.coursera.org	SAFE	0	Domain age: 5,226 days

URL	Verdict	Total Score	Primary Detection Signal
https://www.vietcombank.com.vn	SAFE	0	DNS resolves; no brand mismatch
https://www.vietinbank.vn	SAFE	0	DNS resolves; established domain
https://www.youtube.com	SAFE	0	Domain age: 7,748 days
https://br-icloud.com.br/	SUSPICIOUS	45	Brand "icloud" in non-Apple domain
https://coursere.com	SUSPICIOUS	40	Typosquatting: "coursere" $\approx$ "coursera" (edit distance 1)
https://vietinbank-vn.com	PHISHING	60	Brand "vietinbank" + DNS failure + no WHOIS
https://mbbank-vn.com	PHISHING	60	Brand "mbbank" + DNS failure + no WHOIS

URL	Verdict	Total Score	Primary Detection Signal
https://techcombank-vn.com	PHISHING	55	Brand "techcombank" + 88-day-old domain
http://1.2.3.4/paypal-login	PHISHING	55	IP hostname + HTTP + PayPal brand in path
http://appleid-apple-support.xy z/verify	PHISHING	100	HTTP + suspicious TLD + multiple hyphens + Apple brand + DNS failure

4.6 Implementing Extensions

4.6.1 URL Interception Mechanism

The system uses the `chrome.webRequest.onBeforeRequest` listener to capture the URL at the time the user presses Enter a URL or a domain name in the search bar. It also works the same when the user clicks the link in websites, the interception executes before the webpage content has time to load. The blocked URL is immediately sent to the backend endpoint for verification

4.6.2 Warning Page

When the backend responds that it is an unsafe site: the page is now redirected to the warning.html page (view the UI at appendix - figure A.2).

The warning page includes these information:

- A warning icon and the title "Suspicious Link Blocked".
- A Risk Analysis box that explains the reason why the page is blocked in detail.
- Provide two options: "Back to Safety" or "Proceed Anyway"

4.6.3 Fail-Safe Mode

If the backend server fails or becomes unresponsive, the extension will default to allowing access to avoid interrupting the user's browsing, but will also record the error for maintenance.

4.7 React Native Mobile Application Deployment

Since mobile operating systems like iOS and Android are highly secure and cannot directly interfere with the system browser, this mobile app version only allows users to search in the app and check.

4.7.1 URL Blocking Simulation

The system uses an embedded browser or input field to simulate access behavior.

When a URL is entered, the application calls a Backend API similar to how a Chrome Extension does to ensure consistency of results across platforms.

4.7.2 Warning Modal

Instead of redirecting the page, the mobile application uses a Modal (pop-up window) to warn. Designed to be consistent with the desktop version (view the UI at appendix - figure A.3, figure A.4).

4.7.3 Consistency Verification

Ensure that with the same malicious URL, can two platforms be blocked with the same points and reasons.

Result: the API works stably, with no discrepancies in processing logic between different environments.

CHAPTER 5

DISCUSSION AND EVALUATION

The goal of this chapter is to evaluate the performance of integrated Machine Learning (ML) and Large Language Model (LLM) models, as well as the crucial role of threshold optimization.

5.1. Evaluation metrics of machine learning

5.1.1. Key metrics

The metrics used for evaluation are:

Accuracy: The percentage of total predictions that are correct (both safe and unsafe).

Precision: The percentage of positive predictions (classified as unsafe) that are actually malicious. High precision indicates a low false-positive rate.

Recall (Sensitivity): The percentage of actual phishing URLs that are correctly identified.

High recall is crucial to ensure no malicious websites are missed.

F1-Score: The harmonic average of Precision and Recall, providing a balanced measure of model performance.

ROC Curve and AUC: Used to assess the balance between the true positive rate and the false-positive rate across different classification thresholds.

5.1.2. Machine learning evaluation

Training and testing models: Random Forest, Decision Tree, Support Vector Machine (SVM), Multi-Layer Perceptron (MLP), and XGBoost.

Final Results: XGBoost performs better than other models. So this model was chosen as the primary algorithm for the baseline numerical feature analysis in the hybrid pipeline due to its highest F1-score and AUC.

5.2. Optimize threshold

The system adjusts the threshold to find out the best risk score for classifying phishing and safe websites. The risk score is compiled from various sources, including ML model predictions, rules-based, and LLM analysis.

Initially, the system used a defined threshold, which was 45 points. Even though this threshold could detect phishing URLs, it was not the optimal threshold for detecting more sophisticated phishing cases. To find out the optimal threshold, the system performs testing various threshold levels on the validation dataset. The trained models were iteratively run at each threshold, then evaluated using metrics such as Precision, Recall, and F1-score to find the best balance.

The final results show that the threshold at point 18 is more appropriate than the initial threshold. Lowering the threshold from 45 to 18 significantly increases Recall, which means the system can detect more phishing URLs, especially sophisticated attacks or those with less obvious characteristics. At the same time, the number of false positives remains within acceptable limits, allowing the system to maintain alerting capabilities without causing excessive inconvenience to users.

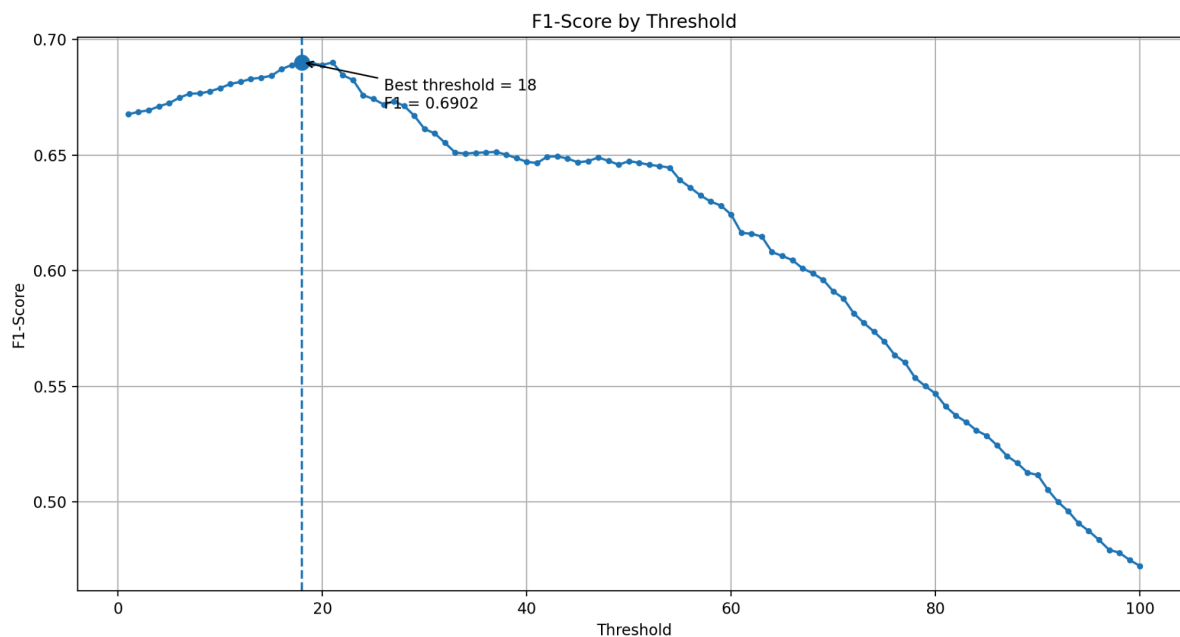


Figure 5.2.1: Threshold visualization

5.3. Evaluate LLM integration

LLM provides semantic and visual analysis to solve the issue that traditional detection systems struggle with. While rule-based and ML detect the URL and website based on features like length, number of special characters, or risk score, LLM analyzes deeper into the content, trade mark, metadata. LLM detects an impersonated brand by comparing the logo or favicon displayed on that page with the actual domain name. If a page includes the brand logo but the domain name does not belong to that brand, the system may consider this as a high-risk indicator. Thanks to the depth analytical capabilities, LLM helps systems reduce both false positives and false negatives. For legitimate websites with poor design, LLM can understand the context to avoid false alarms. Conversely, for sophisticated phishing sites, LLM can detect inconsistencies between logos, content, metadata, and domain names. However, the main limitation of LLM is its higher processing latency compared to traditional rule-based or ML methods. Therefore, LLM is more suitable for in-depth analysis rather than

instantaneous checking of every URL. To address this, the system needs to implement an asynchronous processing architecture, where the fast results from ML and rule-based methods are used for initial alerts, while LLM is activated when more detailed evaluation is needed.

5.4. Discussion about the advantage and disadvantage of hybrid detecting system

The hybrid, multi-layer approach is more effective than using only a single detection method. The ML layer enables the system to analyze URLs quickly and scalably. The machine learning model extracts structure features of URLs then applies models to classify as safe and unsafe. Meanwhile, the LLM layer acts as a mechanism for in-depth analysis. The LLM not only considers numbers or technical characteristics but can also understand text content, logos, images, and the consistency between the displayed brand and the actual domain name. As a result, the system detects sophisticated fake websites better, especially websites that perform brand impersonation, semantic manipulation, or website design that closely resembles legitimate websites. Furthermore, threshold optimization contributes to improved detection efficiency. Moving from an initial threshold which is 45 to an optimized threshold which is 18 indicates that a URL does not need to consider a high risk score as dangerous. Some sophisticated phishing sites may only receive low to average scores, but still contain many suspicious signs. Therefore, a threshold of 18 makes the system more sensitive to potential threats while keeping the number of false positives at an acceptable level. However, the system still has some limitations. Firstly, the use of external LLM APIs like Gemini cause high cost. Free APIs cannot perform too many multiple analysis attempts. Secondly, the system takes more time to analyze images and text to detect a website than a system that only uses rule-based and ML methods to analyze. It increases latency and decreases user experience.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1. Conclusion

This research addresses the increasing prevalence and sophistication of phishing attacks by developing a systematic method for detecting phishing URLs and fake websites across various platforms. Below is a summary of the project's contributions and their significance:

6.1.1. What We Did

We conducted an analysis of a URL and a website based on various techniques such as rules-based heuristic , applied machine learning and LLM to analyze deeply.

6.1.2. What Has Been Accomplished

- Developing a Cross-Platform Phishing Detection System:

We build a hybrid system that includes a multi-layer detection architecture that has been established, combining four main methods: rule-based heuristic, machine learning, large language models, implemented on two client platforms: web in desktop, PC and mobile app.

- Evaluation and Optimization of Machine Learning Models

Selecting the Best Model among five classification algorithms (Decision Tree, Random Forest, SVM, MLP, and XGBoost) were trained and compared on a dataset.

Results: The XGBoost model was selected as the main algorithm for arithmetic feature analysis, because it has the highest F1-score and all other metrics also receive the highest score.

- Refinement of Risk Scoring and Threshold Mechanism

Establish a comprehensive risk score formula that combines scores from rules, ML probabilities, and penalty scores from LLM cross-analysis.

Threshold Calibration:

The threshold has been adjusted from 45 points down to 18 points after testing on the test dataset. Lowering this threshold will increase the Recall metric, ensuring the system can detect more phishing URLs, especially sophisticated links and maintaining an acceptable false positive rate.

- Evaluating the Role of LLM

LLM has been integrated for in-depth analysis, helping to detect brand impersonation and typosquatting by checking for inconsistencies between logos, content, metadata, and actual domain names.

The validated LLM analysis results classify URLs into four categories: legitimate websites, typosquatting, brand impersonation, and maximum risk, demonstrating its ability to detect sophisticated phishing pages that traditional methods struggle to capture.

6.1.3. Why It Matters

This work is critical because the system provides a more flexible defense mechanism. Instead of relying on URL patterns that are easily mimicked by hackers, it focuses on analyzing

features based on the website's trustworthiness. This creates a scalable platform for building real-time security tools such as browser extensions and mobile app detection, protecting users from evolving online threats before they cause any damage.

6.2. Future work

In the future, attack trends will not only increase in URL phishing but also increase in other modern phishing types such as QR code phishing (quishing), CAPTCHA-gated phishing [26]. These new techniques allow attackers to bypass existing security mechanisms by hiding malicious content within QR codes, using CAPTCHAs to deceive users, or combining multiple intermediate steps to conceal the ultimate goal. Therefore, in the future, the system needs to upgrade new modules related to QR code checks and CAPTCHA checks to analyze these attacks more effectively. Specifically, developing a QR code recognition and decoding module will help detect hidden links within QR codes. Behavioral analysis and redirection flow analysis can support the identification of CAPTCHA-gated phishing scenarios. These developments will help the system adapt to new attack trends and enhance user protection.

REFERENCES

- [1] APWG, *Phishing Activity Trends Report: 4th Quarter 2025*. Anti-Phishing Working Group, 2025. [Online]. Available:
https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf
- [2] AVP Suite, “Top 7 Phishing Detection Tools in 2025 for Individuals & Businesses,” 2025. [Online]. Available:
<https://avpsuite.com/blogs/top-7-phishing-detection-tools-in-2025-for-individuals-businesses/>
- [3] M. Khonji, Y. Iraqi, and A. Jones, “Phishing detection: A literature survey,” *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2091–2121, 2013.
- [4] R. Dhamija, J. D. Tygar, and M. Hearst, “Why phishing works,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '06)*, 2006, pp. 581–590.
- [5] R. M. Mohammad, F. Thabtah, and L. McCluskey, “Tutorial and critical analysis of phishing websites methods,” *Computer Science Review*, vol. 17, pp. 1–24, 2015.
- [6] D. K. McGrath and M. Gupta, “Behind phishing: An examination of phisher modi operandi,” in *Proceedings of the 1st USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET '08)*, 2008.
- [7] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious URLs,” in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*, 2009, pp. 1245–1254.

- [8] N. Abdelhamid, A. Ayesh, and F. Thabtah, “Phishing detection based associative classification data mining,” *Expert Systems with Applications*, vol. 41, no. 13, pp. 5948–5959, 2014.
- [9] C. Whittaker, B. Ryner, and M. Nazif, “Large-scale automatic classification of phishing pages,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2010.
- [10] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [11] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [12] F. Ji and D. Kim, “How can we effectively use LLMs for phishing detection?: Evaluating the effectiveness of large language model-based phishing detection models,” *arXiv preprint arXiv:2511.09606*, 2025. [Online]. Available: <https://arxiv.org/html/2511.09606v1>
- [13] F. Trad and A. Chehab, “Large multimodal agents for accurate phishing detection with enhanced token optimization and cost reduction,” *arXiv preprint arXiv:2412.02301*, 2024. [Online]. Available: <https://arxiv.org/html/2412.02301v1>
- [14] Apple Developer, “App Sandbox,” Apple. [Online]. Available: <https://developer.apple.com/documentation/security/app-sandbox>
- [15] Android Developers, “Request app permissions,” Google. [Online]. Available: <https://developer.android.com/training/permissions/requesting?hl=en>
- [16] Google Play Console Help, “Use of permissions,” Google. [Online]. Available: <https://support.google.com/googleplay/android-developer/answer/10144311?hl=en>

[17] “Highly accurate phishing URL detection based on machine learning,” *ResearchGate*.

[Online]. Available:

https://www.researchgate.net/publication/364265241_Highly_accurate_phishing_URL_detection_based_on_machine_learning

[18] Domain Name Wire, “WHOIS privacy and big companies,” 2014. [Online]. Available:

<https://domainnamewire.com/2014/12/04/whois-privacy-big-company/>

[19] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446,

Internet Engineering Task Force, Aug. 2018. [Online]. Available:

<https://www.rfc-editor.org/rfc/rfc8446>

[20] A. Hannousse and S. Yahiouche, “Web page phishing detection,” *Mendeley Data*,

Version 3, Jun. 25, 2021, doi: 10.17632/c2gw7fy2j4.3. [Online]. Available:

<https://data.mendeley.com/datasets/c2gw7fy2j4/3>

[21] S. Tiwari, “Web page Phishing Detection Dataset,” *Kaggle*. [Online]. Available:

<https://www.kaggle.com/datasets/shashwatwork/web-page-phishing-detection-dataset>

[22] IBM, “What is feature engineering?” IBM Think. [Online]. Available:

<https://www.ibm.com/think/topics/feature-engineering>

[23] A. Anand, “Confusion matrix,” *Medium*. [Online]. Available:

<https://medium.com/@atulanand7535/confusion-matrix-809d4798647e>

[24] Google Developers, “Classification: ROC and AUC,” *Machine Learning Crash Course*.

[Online]. Available:

<https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc?hl=v>

i

[25] National Institute of Standards and Technology, *Guide for Conducting Risk Assessments*, NIST Special Publication 800-30 Revision 1, Sep. 2012. [Online]. Available:

<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-30r1.pdf>

[26] KnowBe4, *2026 Phishing Threat Trends Report, Volume 7*. KnowBe4, 2026. [Online].

Available: <https://www.knowbe4.com/resources/reports/2026-phishing-threat-trends-vol-7>

APPENDIX

Threshold	Accuracy	Precision	Recall	F1-Score	TP	FP	TN	FN
30	5.797	5.538	8.206	6.613	4690	3779	1936	1025
31	5.815	5.559	8.101	6.594	4630	3699	2016	1085
32	5.814	5.569	7.958	6.553	4548	3618	2097	1167
33	5.806	5.574	7.829	6.511	4474	3553	2162	1241
34	5.850	5.617	7.732	6.507	4419	3448	2267	1296
35	5.889	5.656	7.668	6.510	4382	3366	2349	1333
36	5.935	5.703	7.589	6.512	4337	3268	2447	1378
37	5.979	5.749	7.515	6.514	4295	3176	2539	1420
38	6.002	5.779	7.433	6.502	4248	3103	2612	1467
39	6.017	5.802	7.356	6.487	4204	3042	2673	1511
40	6.037	5.832	7.267	6.471	4153	2968	2747	1562
41	6.081	5.888	7.171	6.466	4098	2862	2853	1617
42	6.174	5.993	7.083	6.493	4048	2706	3009	1667
43	6.217	6.050	7.010	6.495	4006	2615	3100	1709
44	6.244	6.094	6.929	6.485	3960	2538	3177	1755
45	6.268	6.138	6.836	6.469	3907	2458	3257	1808
46	6.325	6.221	6.747	6.474	3856	2342	3373	1859
47	6.400	6.331	6.658	6.490	3805	2205	3510	1910
48	6.416	6.371	6.583	6.475	3762	2143	3572	1953
49	6.446	6.436	6.483	6.459	3705	2052	3663	2010
50	6.506	6.533	6.416	6.474	3667	1946	3769	2048
51	6.553	6.632	6.310	6.467	3606	1831	3884	2109
52	6.603	6.745	6.194	6.458	3540	1708	4007	2175
53	6.644	6.842	6.107	6.453	3490	1611	4104	2225
54	6.671	6.914	6.037	6.446	3450	1540	4175	2265
55	6.662	6.956	5.913	6.392	3379	1479	4236	2336
56	6.676	7.029	5.808	6.360	3319	1403	4312	2396
57	6.682	7.088	5.711	6.326	3264	1341	4374	2451
58	6.685	7.130	5.641	6.299	3224	1298	4417	2491
59	6.696	7.180	5.584	6.282	3191	1253	4462	2524
60	6.696	7.236	5.489	6.243	3137	1198	4517	2578

Figure A.1: Detection Performance at Key Thresholds

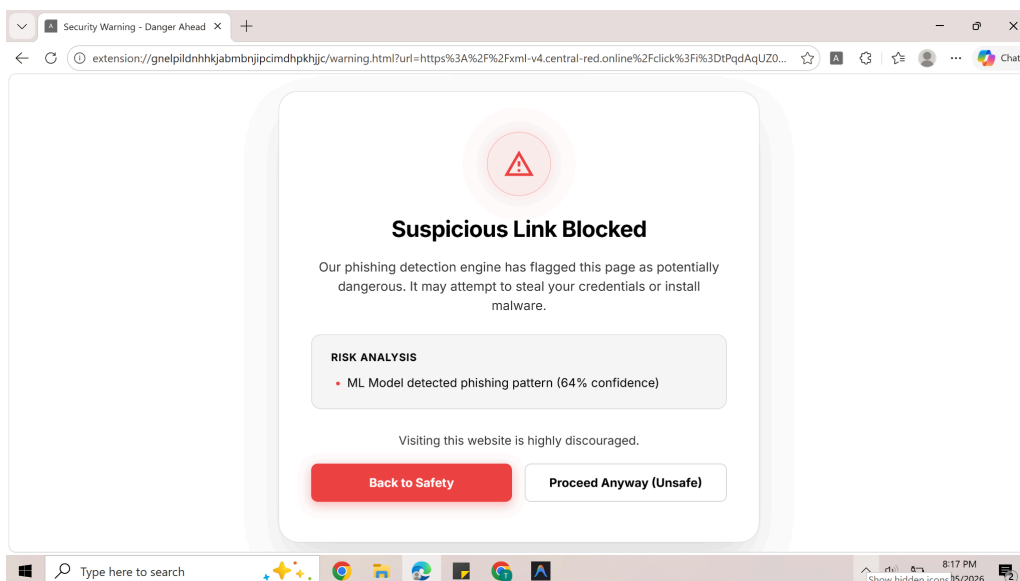


Figure A.2: warning page of web browser

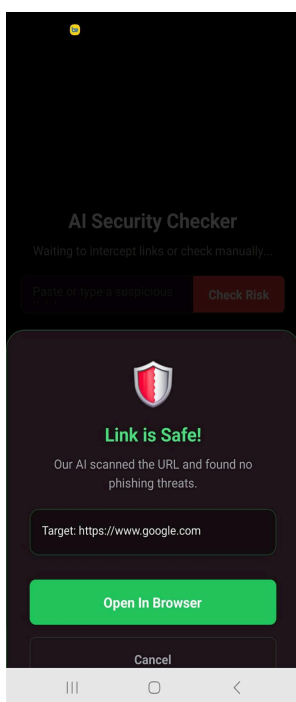


Figure A.3: warning popup for safe URL in mobile

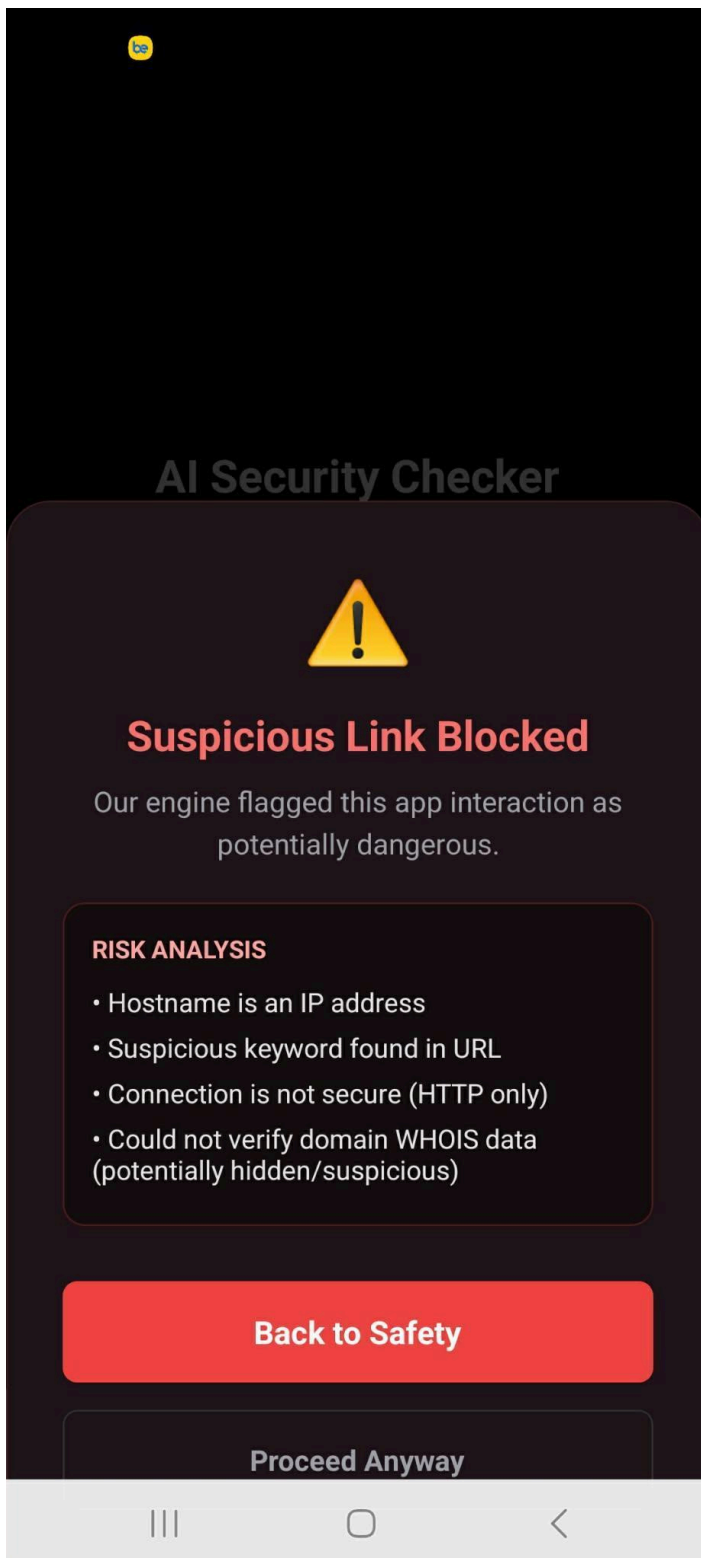


Figure A.4: warning popup for suspicious URL in mobile

